

◇ 1. LINEAR SEARCH (Java)

```
class LinearSearch {  
    public static void main(String[] args) {  
        int[] a = {10, 20, 30, 40, 50};  
        int key = 30;  
        boolean found = false;  
  
        for (int i = 0; i < a.length; i++) {  
            if (a[i] == key) {  
                System.out.println("Element found at index " + i);  
                found = true;  
                break;  
            }  
        }  
  
        if (!found)  
            System.out.println("Element not found");  
    }  
}
```

Working

- Start from first element
 - Compare one by one
 - If match found → stop
-

◇ 2. BINARY SEARCH (Java)

⚠ Array must be sorted

```
class BinarySearch {  
    public static void main(String[] args) {  
        int[] a = {10, 20, 30, 40, 50};  
        int key = 40;  
        int low = 0, high = a.length - 1;  
  
        while (low <= high) {  
            int mid = (low + high) / 2;  
  
            if (a[mid] == key) {  
                System.out.println("Element found at index " + mid);  
                return;  
            } else if (a[mid] < key)  
                low = mid + 1;  
            else  
                high = mid - 1;  
        }  
        System.out.println("Element not found");  
    }  
}
```

🧠 Working

- Check middle element
- If key bigger → go right
- If smaller → go left

◇ 3. BUBBLE SORT (Java)

```
class BubbleSort {  
    public static void main(String[] args) {  
        int[] a = {5, 1, 4, 2, 8};  
  
        for (int i = 0; i < a.length; i++) {  
            for (int j = 0; j < a.length - 1; j++) {  
                if (a[j] > a[j + 1]) {  
                    int temp = a[j];  
                    a[j] = a[j + 1];  
                    a[j + 1] = temp;  
                }  
            }  
        }  
  
        for (int x : a)  
            System.out.print(x + " ");  
    }  
}
```

Working

- Compare adjacent elements
 - Swap if wrong order
 - Largest element goes to end
-

◇ 4. SELECTION SORT (Java)

```
class SelectionSort {  
    public static void main(String[] args) {  
        int[] a = {64, 25, 12, 22, 11};  
        for (int i = 0; i < a.length; i++) {  
            int min = i;  
            for (int j = i + 1; j < a.length; j++) {  
                if (a[j] < a[min])  
                    min = j;  
            }  
            int temp = a[i];  
            a[i] = a[min];  
            a[min] = temp;  
        }  
        for (int x : a)  
            System.out.print(x + " ");  
    }  
}
```

Working

- Find smallest element
- Put it in correct position
- Repeat

◇ 5. INSERTION SORT (Java)

```
class InsertionSort {  
    public static void main(String[] args) {  
        int[] a = {9, 5, 1, 4, 3};  
        for (int i = 1; i < a.length; i++) {  
            int key = a[i];  
            int j = i - 1;  
            while (j >= 0 && a[j] > key) {  
                a[j + 1] = a[j];  
                j--;  
            }  
            a[j + 1] = key;  
        }  
        for (int x : a)  
            System.out.print(x + " ");  
    }  
}
```

Working

- Take one element
- Insert it in proper place
- Like sorting cards

◇ 6. MERGE SORT (Java)

```
class MergeSort {  
    static void merge(int[] a, int l, int m, int r) {  
        int[] temp = new int[r - l + 1];  
        int i = l, j = m + 1, k = 0;  
        while (i <= m && j <= r) {  
            if (a[i] < a[j])  
                temp[k++] = a[i++];  
            else  
                temp[k++] = a[j++];  
        }  
        while (i <= m) temp[k++] = a[i++];  
        while (j <= r) temp[k++] = a[j++];  
        for (i = l, k = 0; i <= r; i++, k++)  
            a[i] = temp[k];  
    }  
    static void mergeSort(int[] a, int l, int r) {  
        if (l < r) {  
            int m = (l + r) / 2;  
            mergeSort(a, l, m);  
            mergeSort(a, m + 1, r);  
            merge(a, l, m, r);  
        }  
    }  
}
```

```

public static void main(String[] args) {
    int[] a = {5, 4, 3, 2, 1};
    mergeSort(a, 0, a.length - 1);
    for (int x : a)
        System.out.print(x + " ");
    }
}

```

Working

- Divide array
- Sort small parts
- Merge them

◇ 7. QUICK SORT (Java)

Code

```

class QuickSort {
    static int partition(int[] a, int low, int high) {
        int pivot = a[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (a[j] < pivot) {
                i++;
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
}

```

```

        int temp = a[i + 1];
        a[i + 1] = a[high];
        a[high] = temp;
        return i + 1;
    }

    static void quickSort(int[] a, int low, int high) {
        if (low < high) {
            int pi = partition(a, low, high);
            quickSort(a, low, pi - 1);
            quickSort(a, pi + 1, high);
        }
    }

    public static void main(String[] args) {
        int[] a = {10, 7, 8, 9, 1};
        quickSort(a, 0, a.length - 1);

        for (int x : a)
            System.out.print(x + " ");
    }
}

```

Working

- Choose pivot
- Smaller left, bigger right
- Repeat recursively

◇ 8. HASHING – MODULO DIVISION + LINEAR PROBING (Java)

✓ Code

```
class HashModulo {  
    public static void main(String[] args) {  
        int size = 10;  
        int[] hashTable = new int[size];  
  
        // initialize with -1  
        for (int i = 0; i < size; i++)  
            hashTable[i] = -1;  
  
        int[] keys = {23, 43, 13, 27};  
  
        for (int key : keys) {  
            int index = key % size;  
  
            // Linear Probing  
            while (hashTable[index] != -1) {  
                index = (index + 1) % size;  
            }  
            hashTable[index] = key;  
        }  
  
        // Display Hash Table  
        for (int i = 0; i < size; i++)  
            System.out.println(i + " -> " + hashTable[i]);  
    }  
}
```

Working (Very Easy)

- Hash function: $\text{key} \% \text{size}$
 - If place is occupied → move **next index**
 - Repeat until empty spot found
-

◇ 9. HASHING – DIGIT EXTRACTION + LINEAR PROBING (Java)

Code

```
class HashDigitExtraction {  
    static int digitExtract(int key) {  
        return (key / 10) % 10; // extract middle digit  
    }  
  
    public static void main(String[] args) {  
        int size = 10;  
        int[] hashTable = new int[size];  
  
        for (int i = 0; i < size; i++)  
            hashTable[i] = -1;  
  
        int[] keys = {123, 456, 789, 145};  
  
        for (int key : keys) {  
            int index = digitExtract(key);  
  
            // Linear Probing  
            while (hashTable[index] != -1) {  
                index = (index + 1) % size;  
            }  
        }  
    }  
}
```

```

        hashTable[index] = key;
    }

    for (int i = 0; i < size; i++)

        System.out.println(i + " -> " + hashTable[i]);
    }
}

```

Working

- Extract a digit from key
- Use it as index
- Collision → move to next index

◇ 10. STACK IMPLEMENTATION (Array – Java)

Code

```

class Stack {

    int top = -1;

    int size = 5;

    int[] stack = new int[size];

    void push(int x) {

        if (top == size - 1)

            System.out.println("Stack Overflow");

        else

            stack[++top] = x;

    }

    void pop() {

        if (top == -1)

```

```

        System.out.println("Stack Underflow");
    else
        System.out.println("Popped: " + stack[top--]);
    }

    void display() {
        for (int i = top; i >= 0; i--)
            System.out.println(stack[i]);
    }

    public static void main(String[] args) {
        Stack s = new Stack();
        s.push(10);
        s.push(20);
        s.push(30);
        s.display();
        s.pop();
    }
}

```

Working

- **LIFO** (Last In First Out)
 - Push → insert at top
 - Pop → remove from top
-

◇ 11. ORDINARY QUEUE (Array – Java)

✓ Code

```
class Queue {  
    int front = 0, rear = -1, size = 5;  
    int[] q = new int[size];  
  
    void enqueue(int x) {  
        if (rear == size - 1)  
            System.out.println("Queue Full");  
        else  
            q[++rear] = x;  
    }  
  
    void dequeue() {  
        if (front > rear)  
            System.out.println("Queue Empty");  
        else  
            System.out.println("Deleted: " + q[front++]);  
    }  
  
    void display() {  
        for (int i = front; i <= rear; i++)  
            System.out.print(q[i] + " ");  
    }  
  
    public static void main(String[] args) {  
        Queue q = new Queue();  
        q.enqueue(10);  
    }  
}
```

```
q.enqueue(20);  
q.enqueue(30);  
q.display();  
q.dequeue();  
}  
}
```

Working

- **FIFO** (First In First Out)
- Insert at rear
- Delete from front

◇ 12. SINGLY LINKED LIST (SLL) – Java

Code

```
class SLL {  
    class Node {  
        int data;  
        Node next;  
        Node(int d) {  
            data = d;  
            next = null;  
        }  
    }  
}  
  
Node head = null;  
  
void insert(int x) {  
    Node n = new Node(x);  
    if (head == null)
```

```

        head = n;
    else {
        Node temp = head;
        while (temp.next != null)
            temp = temp.next;
        temp.next = n;
    }
}

void display() {
    Node temp = head;
    while (temp != null) {
        System.out.print(temp.data + " -> ");
        temp = temp.next;
    }
    System.out.println("NULL");
}

```

```

public static void main(String[] args) {
    SLL list = new SLL();
    list.insert(10);
    list.insert(20);
    list.insert(30);
    list.display();
}
}

```

 **Working**

- Each node has **data + next**

- New node added at end
 - Traverse using next pointer
-

◇ 13. BINARY SEARCH TREE (BST) – Java

Includes **Inorder, Preorder, Postorder, Smallest, Largest, Count**

✓ Code

```
class BST {  
    class Node {  
        int data;  
        Node left, right;  
        Node(int d) {  
            data = d;  
        }  
    }  
  
    Node root = null;  
  
    Node insert(Node r, int x) {  
        if (r == null)  
            return new Node(x);  
        if (x < r.data)  
            r.left = insert(r.left, x);  
        else  
            r.right = insert(r.right, x);  
        return r;  
    }  
  
    void inorder(Node r) {
```



```
    if (r != null) {  
        inorder(r.left);  
        System.out.print(r.data + " ");  
        inorder(r.right);  
    }  
}
```

```
void preorder(Node r) {  
    if (r != null) {  
        System.out.print(r.data + " ");  
        preorder(r.left);  
        preorder(r.right);  
    }  
}
```

```
void postorder(Node r) {  
    if (r != null) {  
        postorder(r.left);  
        postorder(r.right);  
        System.out.print(r.data + " ");  
    }  
}
```

```
int smallest(Node r) {  
    while (r.left != null)  
        r = r.left;  
    return r.data;  
}
```

```
int largest(Node r) {  
    while (r.right != null)  
        r = r.right;  
    return r.data;  
}
```

```
int count(Node r) {  
    if (r == null)  
        return 0;  
    return 1 + count(r.left) + count(r.right);  
}
```

```
public static void main(String[] args) {  
    BST t = new BST();  
    t.root = t.insert(t.root, 50);  
    t.insert(t.root, 30);  
    t.insert(t.root, 70);  
    t.insert(t.root, 20);  
    t.insert(t.root, 40);  
  
    System.out.print("Inorder: ");  
    t.inorder(t.root);  
  
    System.out.print("\nPreorder: ");  
    t.preorder(t.root);  
  
    System.out.print("\nPostorder: ");
```

```

t.postorder(t.root);

System.out.println("\nSmallest: " + t.smallest(t.root));

System.out.println("Largest: " + t.largest(t.root));

System.out.println("Total Nodes: " + t.count(t.root));

}

}

```

Working

- Left < Root < Right
- Traversals visit nodes in different orders
- Smallest = leftmost
- Largest = rightmost

♦ 14. HEAP IMPLEMENTATION (Min Heap & Max Heap) – Java

Code

```

class Heap {

    int size = 0;

    int[] heap = new int[10];

    void insertMax(int x) {

        heap[size] = x;

        int i = size;

        size++;

        while (i > 0 && heap[(i - 1) / 2] < heap[i]) {

            int temp = heap[i];

            heap[i] = heap[(i - 1) / 2];

            heap[(i - 1) / 2] = temp;

```

```
        i = (i - 1) / 2;
    }
}
```

```
void display() {
    for (int i = 0; i < size; i++)
        System.out.print(heap[i] + " ");
}
```

```
public static void main(String[] args) {
    Heap h = new Heap();
    h.insertMax(10);
    h.insertMax(40);
    h.insertMax(20);
    h.insertMax(30);
    h.display();
}
}
```

Working

- Heap stored in array
 - Parent always larger (MaxHeap)
 - Insert and adjust upward
-

◇ 15. GRAPH – DFS (Java)

✓ Code

```
import java.util.*;

class DFS {

    static void dfs(int v, boolean[] visited, int[][] graph) {

        visited[v] = true;

        System.out.print(v + " ");

        for (int i = 0; i < graph.length; i++) {

            if (graph[v][i] == 1 && !visited[i])

                dfs(i, visited, graph);

        }

    }

    public static void main(String[] args) {

        int[][] graph = {

            {0,1,1,0},

            {1,0,0,1},

            {1,0,0,1},

            {0,1,1,0}

        };

        boolean[] visited = new boolean[4];

        dfs(0, visited, graph);

    }

}
```

Working

- Go **deep first**
 - Visit node
 - Recursively visit neighbors
-

◇ 16. GRAPH – BFS (Java)

Code

```
import java.util.*;
```

```
class BFS {  
    public static void main(String[] args) {  
        int[][] graph = {  
            {0,1,1,0},  
            {1,0,0,1},  
            {1,0,0,1},  
            {0,1,1,0}  
        };  
  
        boolean[] visited = new boolean[4];  
        Queue<Integer> q = new LinkedList<>();  
  
        q.add(0);  
        visited[0] = true;  
  
        while (!q.isEmpty()) {  
            int v = q.poll();  
            System.out.print(v + " ");  
        }  
    }  
}
```

```
for (int i = 0; i < 4; i++) {  
    if (graph[v][i] == 1 && !visited[i]) {  
        q.add(i);  
        visited[i] = true;  
    }  
}  
}  
}
```

Working

- Visit **level by level**
- Use queue
- FIFO order